

API Smoke Test - Live Claude API

3 queries x Strategy A vs B against the running Flask server

■API Smoke Test - Live Claude API

Run: 2026-05-14 01:32:17

Endpoint: POST http://127.0.0.1:5000/chat/ask

Provider: anthropic / claude-haiku-4-5-20251001

Health: {"chats":0,"documents":{"bytes":3178,"chunks":8,"docs":8},"langfuse":false,"status":"ok"}

QUERY: How does the system handle peak load?

STRATEGY: A

LATENCY: 2471.96 ms (embed+expand 91.34ms / search 0.23ms / generate 2380.29ms)

ANSWER:

According to the provided context, the system handles peak load through the following mechanisms:

1. **Horizontal auto-scaling with Kubernetes HPA**: New pods are automatically spun up from a warm pool when CPU utilization crosses 70%, or request latency p95 exceeds 300ms.
2. **Predictive scaling**: The system pre-provisions capacity in advance during known traffic spikes, such as marketing campaigns or product launches.
3. **Connection draining**: During scale-down, in-flight requests are allowed to complete before pods are terminated.

SOURCES:

#1 auto_scaling.txt sim=0.5135

The system handles peak load through horizontal auto-scaling powered by Kubernetes HPA. When CPU utilization crosses 70%, or request latency p95 exceeds 300ms, new pods are automatically spun up from a warm pool.

#2 load_balancer.txt sim=0.3939

Incoming traffic is distributed across regional load balancers operating at both L4 and L7. The L7 layer uses weighted round-robin to route traffic to the healthiest pods.

#3 rate_limiting.txt sim=0.3641

APIs are protected by a tiered rate-limiting system using token-bucket and leaky-bucket algorithms. Anonymous requests are capped at 60 requests per minute per IP, while authenticated users have a higher limit.

QUERY: How does the system handle peak load?

STRATEGY: B

EXPANDED: How does the system handle peak load? peak traffic management system scalability high traffic load balanced

LATENCY: 14179.77 ms (embed+expand 7683.75ms / search 0.09ms / generate 6495.87ms)

ANSWER:

According to the provided context, the system handles peak load through the following mechanisms:

1. **Horizontal Auto-Scaling**: The system uses Kubernetes HPA (Horizontal Pod Autoscaler) to spin up new pods when CPU utilization crosses 70%, or request latency p95 exceeds 300ms.
2. **Predictive Scaling**: A predictive scaler pre-provisions capacity during known traffic spikes, such as marketing campaigns or product launches.
3. **Connection Draining**: During scale-down, connection draining ensures that in-flight requests complete before pods are terminated.

SOURCES:

#1 auto_scaling.txt sim=0.5127

The system handles peak load through horizontal auto-scaling powered by Kubernetes HPA. When CPU utilization crosses 70%, or request latency p95 exceeds 300ms, new pods are automatically spun up from a warm pool.

#2 rate_limiting.txt sim=0.4238

APIs are protected by a tiered rate-limiting system using token-bucket and leaky-bucket algorithms. Anonymous requests are capped at 60 requests per minute per IP, while authenticated users have a higher limit.

#3 load_balancer.txt sim=0.4234

Incoming traffic is distributed across regional load balancers operating at both L4 and L7. The L7 layer uses weighted round-robin to route traffic to the healthiest pods.

QUERY: What happens when too many users hit the API at once?

STRATEGY: A

LATENCY: 3834.64 ms (embed+expand 22.6ms / search 0.09ms / generate 3811.89ms)

ANSWER:

Based on the provided context, when too many users hit the API at once, several mechanisms work together to handle the load:

1. **Rate Limiting**: Anonymous traffic is capped at 60 requests per minute per IP, while authenticated users have a higher limit.
2. **Burst Credits**: The rate-limiting system uses token-bucket and leaky-bucket algorithms with burst credits to handle temporary spikes in traffic.

3. ****Auto-Scaling****: The system uses Kubernetes HPA to horizontally scale by spinning up new pods from a warm pool.
 4. ****Load Balancing****: Incoming traffic is distributed across regional load balancers at both L4 and L7 layers.
- These mechanisms work together to manage sudden increases in user traffic and prevent system overload.

SOURCES:

- #1 rate_limiting.txt sim=0.6124
APIs are protected by a tiered rate-limiting system using token-bucket and leaky-bucket algorithms. Anonymous traffic is capped at 60 requests per minute per IP, while authenticated users can burst up to 100 requests per minute.
- #2 load_balancer.txt sim=0.2687
Incoming traffic is distributed across regional load balancers operating at both L4 and L7. The L7 layer uses weighted round-robin for distribution.
- #3 auto_scaling.txt sim=0.2509
The system handles peak load through horizontal auto-scaling powered by Kubernetes HPA. When CPU utilization exceeds 80%, new pods are spun up from a warm pool.

QUERY: What happens when too many users hit the API at once?

STRATEGY: B

EXPANDED: What happens when too many concurrent users simultaneously request the API endpoint, causing high traffic and potential system overload?

LATENCY: 5924.54 ms (embed+expand 3166.3ms / search 0.09ms / generate 2758.07ms)

ANSWER:

Based on the provided context, when too many users hit the API at once, several mechanisms work together:

1. ****Rate Limiting****: Anonymous traffic is capped at 60 requests per minute per IP, while authenticated users can burst up to 100 requests per minute.
2. ****Burst Absorption****: Burst credits accumulate in the token-bucket and leaky-bucket algorithms to absorb sudden spikes in traffic.
3. ****Auto-scaling****: The system handles peak load through horizontal auto-scaling via Kubernetes HPA. When CPU utilization exceeds 80%, new pods are spun up from a warm pool.
4. ****Load Distribution****: Incoming traffic is distributed across regional load balancers operating at L4 and L7 layers.

SOURCES:

- #1 rate_limiting.txt sim=0.6701
APIs are protected by a tiered rate-limiting system using token-bucket and leaky-bucket algorithms. Anonymous traffic is capped at 60 requests per minute per IP, while authenticated users can burst up to 100 requests per minute.
- #2 load_balancer.txt sim=0.3916
Incoming traffic is distributed across regional load balancers operating at both L4 and L7. The L7 layer uses weighted round-robin for distribution.
- #3 auto_scaling.txt sim=0.3862
The system handles peak load through horizontal auto-scaling powered by Kubernetes HPA. When CPU utilization exceeds 80%, new pods are spun up from a warm pool.

QUERY: Explain the failover mechanism during outages.

STRATEGY: A

LATENCY: 2805.82 ms (embed+expand 30.94ms / search 0.08ms / generate 2774.75ms)

ANSWER:

Based on the provided context, here is the failover mechanism during outages:

The primary database operates in a ****multi-AZ configuration with synchronous replication**** to a standby replica in a different Availability Zone. Additionally, the system maintains ****cross-region replication with warm standbys**** as part of disaster recovery. At the load balancing layer, ****health checks run at 5-second intervals**** to detect unhealthy backends and remove them from the rotation.

SOURCES:

- #1 database_failover.txt sim=0.5181
The primary database operates in a multi-AZ configuration with synchronous replication to a standby replica in a different Availability Zone.
- #2 disaster_recovery.txt sim=0.4372
Disaster recovery procedures define an RPO of 5 minutes and an RTO of 30 minutes. Cross-region replication is maintained for disaster recovery.
- #3 load_balancer.txt sim=0.3599
Incoming traffic is distributed across regional load balancers operating at both L4 and L7. The L7 layer uses weighted round-robin for distribution.

QUERY: Explain the failover mechanism during outages.

STRATEGY: B

EXPANDED: failover mechanism outage recovery high availability redundancy switching backup systems disaster recovery

LATENCY: 6726.04 ms (embed+expand 2754.66ms / search 0.12ms / generate 3971.22ms)

ANSWER:

Failover Mechanism During Outages

Based on the provided context, the failover mechanism operates at multiple levels:

Database Level

The primary database uses a **multi-AZ configuration with synchronous replication** to a standby replica in a

Load Balancer Level

Regional load balancers perform **health checks at 5-second intervals** to detect unhealthy backends. Unhealth

Incident Coordination

Beyond technical failover, **incident commanders coordinate response via a runbook playbook with auto-paging t**

Overall Recovery Targets

The disaster recovery procedures define an **RPO of 5 minutes and an RTO of 30 minutes**, with **cross-region**

SOURCES:

#1 disaster_recovery.txt sim=0.6117

Disaster recovery procedures define an RPO of 5 minutes and an RTO of 30 minutes. Cross-region replicati

#2 database_failover.txt sim=0.5664

The primary database operates in a multi-AZ configuration with synchronous replication to a standby repl

#3 load_balancer.txt sim=0.271

Incoming traffic is distributed across regional load balancers operating at both L4 and L7. The L7 layer